Lekcja 06: Kolekcje II – Słowniki, zbiory i krotki

`#lekcja` `#datascience` `#python` `#kolekcje` `#slowniki` `#zbiory` `#krotki`

W tej lekcji poznamy trzy kolejne kluczowe kolekcje w Pythonie: **słowniki (dictionaries)**, **zbiory (sets)** oraz **krotki (tuples)**. Każda z tych struktur danych ma swoje unikalne cechy i zastosowania w Data Science – od przechowywania par klucz-wartość po operacje na zbiorach unikalnych elementów. Zrozumienie tych typów pozwoli Ci efektywnie przetwarzać i organizować dane w projektach analitycznych.

---

1. Słowniki (Dictionaries)

> **Definition**
>
> **Słownik** to mutowalny, nieuporządkowany zbiór par **klucz-wartość** (key-value pairs). Klucze muszą być unikalne i niemutowalne (najczęściej stringi lub liczby), a wartości mogą być dowolnego typu.

Wprowadzenie

Słowniki to jedna z najważniejszych struktur danych w Pythonie i Data Science. W przeciwieństwie do list, gdzie dostęp do elementów odbywa się przez indeksy numeryczne, w słownikach używamy **kluczy**, co czyni je idealnym narzędziem do przechowywania danych strukturalnych, takich jak atrybuty obiektów, konfiguracje, dane JSON czy statystyki.

Zastosowania w Data Science:

- Przechowywanie metadanych o datasetach (nazwa, rozmiar, typ)
- Mapowanie kategorii na wartości numeryczne
- Liczenie częstości występowania elementów
- Przechowywanie wyników eksperymentów ML
- Reprezentacja rekordów danych (jak w JSON)

Przykład 1: Tworzenie i podstawowe operacje na słownikach

```
# Tworzenie pustego słownika
pusty_slownik = {}
inny_pusty = dict()
```

```python
# Tworzenie słownika z danymi
student = {
    'imie': 'Anna',
    'wiek': 25,
    'oceny': [4.5, 5.0, 4.0],
    'aktywny': True
}

# Dostęp do wartości przez klucz
print(student['imie'])  # Anna
print(student['wiek'])   # 25

# Dostęp z użyciem metody get() - bezpieczniejsze (zwraca None jeśli klucz
nie istnieje)
print(student.get('imie'))        # Anna
print(student.get('miasto'))      # None (zamiast KeyError)
print(student.get('miasto', 'Warszawa'))  # Warszawa (wartość domyślna)

# Dodawanie nowych par klucz-wartość
student['miasto'] = 'Kraków'
student['email'] = 'anna@example.com'

# Modyfikacja wartości
student['wiek'] = 26

# Usuwanie elementów
del student['aktywny']  # Usuwa parę klucz-wartość
wiek_studenta = student.pop('wiek')  # Usuwa i zwraca wartość (26)

print(student)
# {'imie': 'Anna', 'oceny': [4.5, 5.0, 4.0], 'miasto': 'Kraków', 'email':
'anna@example.com'}
```

Wyjaśnienie:

- Słowniki tworzymy używając nawiasów klamrowych `{}` lub funkcji `dict()`
- Dostęp do wartości: `slownik[klucz]` lub `slownik.get(klucz)`
- Metoda `.get()` jest bezpieczniejsza - nie wywołuje błędu gdy klucz nie istnieje
- Słowniki są mutowalne - możemy dodawać, modyfikować i usuwać elementy

Przykład 2: Iteracja po słownikach i metody słowników

```python
# Dataset z wynikami eksperymentu ML
wyniki_modelu = {
```

```python
    'accuracy': 0.95,
    'precision': 0.92,
    'recall': 0.89,
    'f1_score': 0.90,
    'training_time': 45.3
}

# Iteracja po kluczach (domyślnie)
print("Metryki modelu:")
for metryka in wyniki_modelu:
    print(f"- {metryka}")

# Iteracja po kluczach (jawnie)
for klucz in wyniki_modelu.keys():
    print(klucz)

# Iteracja po wartościach
print("\nWartości metryk:")
for wartosc in wyniki_modelu.values():
    print(wartosc)

# Iteracja po parach klucz-wartość (najczęściej używane!)
print("\nRaport z modelu:")
for metryka, wartosc in wyniki_modelu.items():
    if metryka == 'training_time':
        print(f"{metryka}: {wartosc:.1f} sekund")
    else:
        print(f"{metryka}: {wartosc:.2%}")

# Sprawdzanie czy klucz istnieje
if 'accuracy' in wyniki_modelu:
    print(f"\nAccuracy modelu: {wyniki_modelu['accuracy']:.2%}")

if 'loss' not in wyniki_modelu:
    print("Brak informacji o loss function")

# Liczba elementów w słowniku
print(f"\nLiczba metryk: {len(wyniki_modelu)}")

# Pobieranie wszystkich kluczy, wartości, par
klucze = list(wyniki_modelu.keys())
wartosci = list(wyniki_modelu.values())
pary = list(wyniki_modelu.items())

print(f"Klucze: {klucze}")
print(f"Wartości: {wartosci}")
```

Wyjaśnienie:

- `.keys()` - zwraca wszystkie klucze
- `.values()` - zwraca wszystkie wartości
- `.items()` - zwraca pary (klucz, wartość) jako tuple
- `in` operator sprawdza czy klucz istnieje w słowniku
- Iteracja po `.items()` jest bardzo wygodna w Data Science do raportowania

Przykład 3: Liczenie częstości - zastosowanie praktyczne w DS

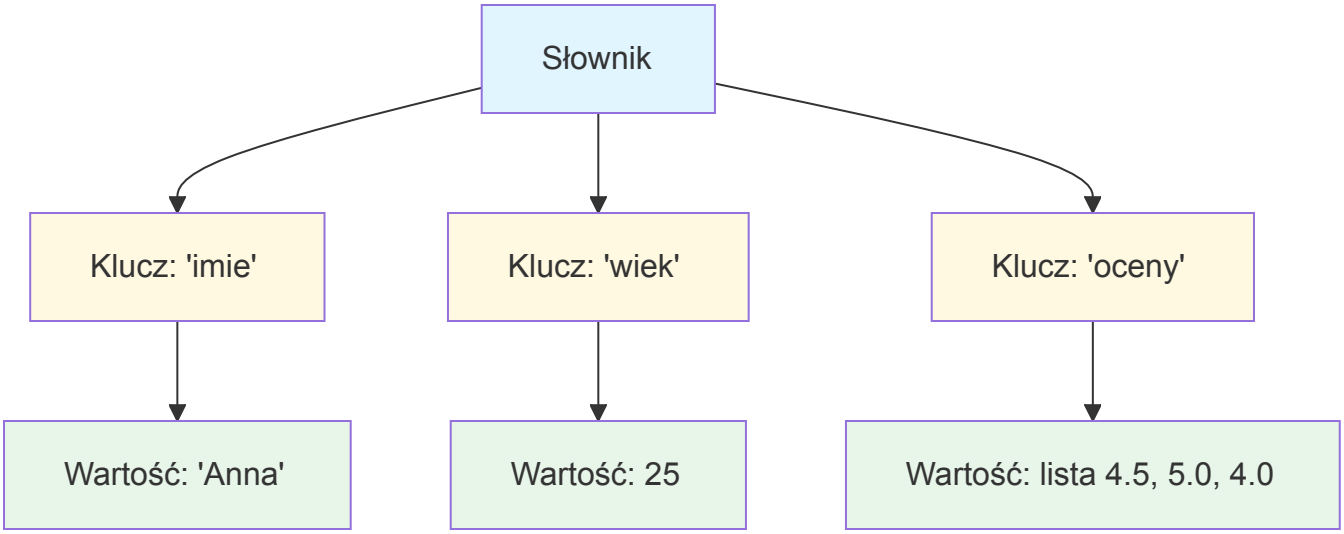
```
# Analiza częstości występowania kategorii w danych
# Symulacja datasetu: wyniki ankiety o preferowanych językach programowania

odpowiedzi = [
    'Python', 'Java', 'Python', 'JavaScript', 'Python',
    'C++', 'Python', 'Java', 'JavaScript', 'Python',
    'Python', 'R', 'Python', 'Julia', 'Python',
    'Java', 'Python', 'Python', 'JavaScript', 'Python'
]

# Sposób 1: Ręczne liczenie z użyciem słownika
czestosci = {}
for jezyk in odpowiedzi:
    if jezyk in czestosci:
        czestosci[jezyk] += 1
    else:
        czestosci[jezyk] = 1

print("Częstości języków programowania:")
for jezyk, liczba in czestosci.items():
    procent = (liczba / len(odpowiedzi)) * 100
    print(f"{jezyk}: {liczba} odpowiedzi ({procent:.1f}%)")

# Sposób 2: Użycie metody .get() z wartością domyślną (eleganckie!)
czestosci_v2 = {}
for jezyk in odpowiedzi:
    czestosci_v2[jezyk] = czestosci_v2.get(jezyk, 0) + 1

# Sposób 3: Użycie metody .setdefault()
czestosci_v3 = {}
for jezyk in odpowiedzi:
    czestosci_v3.setdefault(jezyk, 0)
    czestosci_v3[jezyk] += 1

# Znajdowanie najpopularniejszego języka
```

```
najpopularniejszy = max(czestosci.items(), key=lambda x: x[1])
print(f"\nNajpopularniejszy język: {najpopularniejszy[0]}
({najpopularniejszy[1]} głosów)")

# Sortowanie wyników malejąco
posortowane = sorted(czestosci.items(), key=lambda x: x[1], reverse=True)
print("\nRanking języków:")
for pozycja, (jezyk, liczba) in enumerate(posortowane, 1):
    print(f"{pozycja}. {jezyk}: {liczba} głosów")
```

Wyjaśnienie:

- Słowniki idealnie nadają się do zliczania częstości elementów
- Metoda `.get(klucz, domyslna)` pozwala uniknąć sprawdzania czy klucz istnieje
- Funkcja `max()` z parametrem `key` znajduje element o maksymalnej wartości
- `sorted()` z `key=lambda` pozwala sortować słowniki po wartościach
- To częsty pattern w analizie danych kategorycznych

Wizualizacja struktury słownika



Zaawansowane operacje na słownikach

```
# Łączenie słowników (Python 3.9+)
metadane1 = {'nazwa': 'iris_dataset', 'rozmiar': 150}
metadane2 = {'typ': 'klasyfikacja', 'klasy': 3}
polaczone = metadane1 | metadane2  # Operator | (union)
print(polaczone)
# {'nazwa': 'iris_dataset', 'rozmiar': 150, 'typ': 'klasyfikacja', 'klasy':
3}

# Aktualizacja słownika innym słownikiem
```

```python
config = {'learning_rate': 0.01, 'epochs': 100}
nowa_config = {'epochs': 150, 'batch_size': 32}
config.update(nowa_config)
print(config)
# {'learning_rate': 0.01, 'epochs': 150, 'batch_size': 32}

# Dictionary comprehension - tworzenie słowników w jednej linii
liczby = [1, 2, 3, 4, 5]
kwadraty = {x: x**2 for x in liczby}
print(kwadraty)  # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Filtrowanie podczas tworzenia
parzyste_kwadraty = {x: x**2 for x in liczby if x % 2 == 0}
print(parzyste_kwadraty)  # {2: 4, 4: 16}

# Czyszczenie słownika
kwadraty.clear()
print(kwadraty)  # {}
```

Tip

Dictionary comprehension to elegancki sposób na tworzenie słowników w jednej linii. Jest szybsze i bardziej czytelne niż pętle for przy prostych transformacjach.

Warning

Słowniki przed Pythonem 3.7 **nie zachowywały kolejności** dodawania elementów. Od Pythona 3.7+ kolejność jest gwarantowana, co ułatwia pracę z danymi sekwencyjnymi.

Success

W Data Science często używamy słowników do:

- Przechowywania hyperparametrów modeli
- Mapowania kategorii na liczby (label encoding)
- Agregacji wyników eksperymentów
- Reprezentacji JSON API responses

2. Zbiory (Sets)

Definition

Zbiór (set) to mutowalny, **nieuporządkowany** zbiór **unikalnych** elementów. Zbiory nie dopuszczają duplikatów i nie zachowują kolejności dodawania elementów.

Wprowadzenie

Zbiory (sets) w Pythonie są implementacją matematycznych zbiorów i obsługują operacje takie jak suma, przecięcie, różnica czy różnica symetryczna. Są niezwykle przydatne gdy potrzebujemy:

- Usunąć duplikaty z danych
- Sprawdzić czy element należy do zbioru (bardzo szybkie!)
- Wykonać operacje na zbiorach (np. znaleźć wspólne elementy)

Zastosowania w Data Science:

- Deduplikacja danych (usuwanie duplikatów)
- Znajdowanie unikalnych wartości w kolumnach
- Porównywanie grup danych (wspólne/różniące się elementy)
- Walidacja danych (sprawdzanie czy wartości są w dozwolonym zbiorze)

Przykład 1: Tworzenie zbiorów i podstawowe operacje

```
# Tworzenie zbiorów
pusty_zbior = set()  # Uwaga: {} tworzy słownik, nie zbiór!
liczby = {1, 2, 3, 4, 5}
owoce = {'jabłko', 'banan', 'pomarańcza'}

# Zbiór z listy (automatycznie usuwa duplikaty!)
lista_z_duplikatami = [1, 2, 2, 3, 3, 3, 4, 5, 5]
zbior_unikalnych = set(lista_z_duplikatami)
print(zbior_unikalnych)  # {1, 2, 3, 4, 5}

# Długość zbioru
print(f"Liczba unikalnych elementów: {len(zbior_unikalnych)}")

# Sprawdzanie czy element należy do zbioru (BARDZO SZYBKIE - O(1))
print(3 in zbior_unikalnych)     # True
print(10 in zbior_unikalnych)    # False
```

```python
# Dodawanie elementów
liczby.add(6)
liczby.add(3)  # Nie doda duplikatu!
print(liczby)  # {1, 2, 3, 4, 5, 6}

# Dodawanie wielu elementów
liczby.update([7, 8, 9])
print(liczby)  # {1, 2, 3, 4, 5, 6, 7, 8, 9}

# Usuwanie elementów
liczby.remove(9)      # Wywołuje KeyError jeśli element nie istnieje
liczby.discard(100)   # Nie wywołuje błędu jeśli element nie istnieje
usuniety = liczby.pop()  # Usuwa i zwraca losowy element
print(f"Usunięto: {usuniety}")
print(liczby)
```

Wyjaśnienie:

- Zbiory tworzymy używając `{}` z elementami lub `set()`
- Pusty zbiór: **zawsze** `set()`, nigdy `{}` (to tworzy słownik!)
- `.add()` dodaje pojedynczy element
- `.update()` dodaje wiele elementów
- `.remove()` vs `.discard()` - pierwszy wywołuje błąd przy braku elementu
- Sprawdzanie przynależności (`in`) jest **bardzo szybkie** w zbiorach

Przykład 2: Operacje na zbiorach - matematyka w praktyce

```python
# Symulacja danych: użytkownicy którzy kupili produkty
kupili_laptopy = {'Anna', 'Jan', 'Marek', 'Zofia', 'Piotr'}
kupili_telefony = {'Jan', 'Ewa', 'Zofia', 'Tomasz'}
kupili_tablety = {'Marek', 'Zofia', 'Ewa'}

# SUMA (union) - użytkownicy którzy kupili cokolwiek
wszyscy_kupujacy = kupili_laptopy | kupili_telefony | kupili_tablety
print(f"Wszyscy kupujący: {wszyscy_kupujacy}")
# lub: kupili_laptopy.union(kupili_telefony, kupili_tablety)

# PRZECIĘCIE (intersection) - użytkownicy którzy kupili laptopy I telefony
laptop_i_telefon = kupili_laptopy & kupili_telefony
print(f"Kupili laptop i telefon: {laptop_i_telefon}")
# lub: kupili_laptopy.intersection(kupili_telefony)

# Wszyscy którzy kupili wszystkie 3 produkty
wszystkie_3 = kupili_laptopy & kupili_telefony & kupili_tablety
```

```
print(f"Kupili wszystkie 3 produkty: {wszystkie_3}")

# RÓŻNICA (difference) - kupili laptopy ale NIE telefony
tylko_laptopy = kupili_laptopy - kupili_telefony
print(f"Tylko laptopy (bez telefonów): {tylko_laptopy}")
# lub: kupili_laptopy.difference(kupili_telefony)

# RÓŻNICA SYMETRYCZNA (symmetric difference) - kupili laptop LUB telefon,
ale nie oba
laptop_xor_telefon = kupili_laptopy ^ kupili_telefony
print(f"Laptop XOR telefon: {laptop_xor_telefon}")
# lub: kupili_laptopy.symmetric_difference(kupili_telefony)

# Sprawdzanie relacji między zbiorami
zestaw1 = {1, 2, 3}
zestaw2 = {1, 2, 3, 4, 5}
zestaw3 = {6, 7, 8}

# Czy zestaw1 jest podzbiorem zestaw2?
print(zestaw1.issubset(zestaw2))    # True (zestaw1 ⊆ zestaw2)
print(zestaw1 <= zestaw2)            # True (alternatywna notacja)

# Czy zestaw2 jest nadzbiorem zestaw1?
print(zestaw2.issuperset(zestaw1))  # True (zestaw2 ⊇ zestaw1)
print(zestaw2 >= zestaw1)            # True

# Czy zbiory są rozłączne (bez wspólnych elementów)?
print(zestaw1.isdisjoint(zestaw3))  # True (brak wspólnych elementów)
print(zestaw1.isdisjoint(zestaw2))  # False (mają wspólne elementy)
```

Wyjaśnienie:

- `|` lub `.union()` - suma zbiorów
- `&` lub `.intersection()` - część wspólna
- `-` lub `.difference()` - różnica zbiorów
- `^` lub `.symmetric_difference()` - różnica symetryczna (elementy tylko w jednym ze zbiorów)
- Relacje: `.issubset()`, `.issuperset()`, `.isdisjoint()`

Przykład 3: Deduplikacja i analiza danych kategorycznych

```
# Symulacja datasetu: kategorie produktów z duplikatami
dane_transakcji = [
    'Elektronika', 'AGD', 'Elektronika', 'Moda', 'Sport',
```

```
    'AGD', 'Elektronika', 'Moda', 'Sport', 'Książki',
    'Sport', 'Moda', 'Elektronika', 'Książki', 'AGD',
    'Elektronika', 'Sport', 'Moda', 'AGD', 'Elektronika'
]

# Znajdowanie unikalnych kategorii
unikalne_kategorie = set(dane_transakcji)
print(f"Unikalne kategorie: {unikalne_kategorie}")
print(f"Liczba unikalnych kategorii: {len(unikalne_kategorie)}")

# Porównanie - ile duplikatów?
print(f"Wszystkie transakcje: {len(dane_transakcji)}")
print(f"Unikalne kategorie: {len(unikalne_kategorie)}")
print(f"Duplikaty: {len(dane_transakcji) - len(unikalne_kategorie)}")

# Walidacja danych - sprawdzanie czy wartości są w dozwolonym zbiorze
dozwolone_kategorie = {'Elektronika', 'AGD', 'Moda', 'Sport', 'Książki',
'Dom'}

nowe_dane = ['Elektronika', 'Meble', 'Sport', 'XYZ']  # Zawiera błędne
kategorie

nieprawidlowe = []
for kategoria in nowe_dane:
    if kategoria not in dozwolone_kategorie:
        nieprawidlowe.append(kategoria)

print(f"Nieprawidłowe kategorie: {nieprawidlowe}")

# Użycie zbiorów do znalezienia nieprawidłowych kategorii (eleganckie!)
zbior_nowych = set(nowe_dane)
nieprawidlowe_zbior = zbior_nowych - dozwolone_kategorie
print(f"Nieprawidłowe kategorie (set): {nieprawidlowe_zbior}")

# Analiza - które kategorie się pojawiły w obu grupach danych?
dane_wczoraj = ['Elektronika', 'AGD', 'Sport', 'Dom']
dane_dzisiaj = ['Moda', 'Sport', 'Książki', 'Elektronika']

wspolne = set(dane_wczoraj) & set(dane_dzisiaj)
tylko_wczoraj = set(dane_wczoraj) - set(dane_dzisiaj)
tylko_dzisiaj = set(dane_dzisiaj) - set(dane_wczoraj)

print(f"\nKategorie w obu dniach: {wspolne}")
print(f"Tylko wczoraj: {tylko_wczoraj}")
print(f"Tylko dzisiaj: {tylko_dzisiaj}")
```

Wyjaśnienie:

- Zbiory automatycznie usuwają duplikaty - idealne do znajdowania unikalnych wartości
- Sprawdzanie `in` jest bardzo szybkie w zbiorach (szybsze niż w listach!)
- Operacje zbiorowe pozwalają łatwo porównywać grupy danych
- Walidacja danych - sprawdzanie czy wartości należą do dozwolonego zakresu

Wizualizacja operacji na zbiorach

```
Error parsing Mermaid diagram!

Parse error on line 5:
...: 3,4,5,6]    C[A | B Union: 1,2,3,4,5
--------------------^
Expecting 'SQE', 'TAGEND', 'UNICODE_TEXT', 'TEXT', 'TAGSTART', got 'PIPE'
```

Tip

Zbiory są **niezmiernie szybkie** przy sprawdzaniu czy element należy do kolekcji. Jeśli masz dużą listę i często sprawdzasz `element in lista`, rozważ konwersję do zbioru: `element in set(lista)`.

Warning

Elementy zbioru muszą być **niemutowalne** (hashable). Możesz mieć zbiór liczb, stringów, tuple, ale **nie możesz** mieć zbioru list czy słowników.

Note

Istnieje też **frozenset** - niemutowalny odpowiednik zbioru. Można go użyć jako klucza słownika lub elementu innego zbioru.

---

3. Krotki (Tuples)

Definition

Krotka (tuple) to **niemutowalna**, uporządkowana sekwencja elementów. Po utworzeniu nie można zmienić jej zawartości (dodawać, usuwać, modyfikować elementów).

Wprowadzenie

Krotki są podobne do list, ale mają jedną kluczową różnicę: są **niemutowalne**. To sprawia, że są szybsze, bezpieczniejsze i mogą być użyte jako klucze słowników. Niemutowalność gwarantuje, że dane nie zostaną przypadkowo zmienione.

Zastosowania w Data Science:

- Przechowywanie współrzędnych (x, y) lub (lat, lon)
- Zwracanie wielu wartości z funkcji
- Reprezentacja rekordów danych które nie powinny się zmieniać
- Klucze słowników (gdy potrzebujemy złożonych kluczy)
- Struktury danych które nie powinny być modyfikowane

Przykład 1: Tworzenie krotek i podstawowe operacje

```
# Tworzenie krotek
pusta_krotka = ()
jednoelementowa = (5,)  # Uwaga: przecinek jest wymagany!
bez_nawiasow = 1, 2, 3  # Również tworzy krotkę

# Różne typy danych w krotce
dane_osoby = ('Anna', 25, 1.65, True)
print(dane_osoby)

# Dostęp do elementów (jak w listach)
imie = dane_osoby[0]
wiek = dane_osoby[1]
print(f"{imie} ma {wiek} lat")

# Slicing (wycinki)
print(dane_osoby[1:3])   # (25, 1.65)
print(dane_osoby[:2])    # ('Anna', 25)
print(dane_osoby[-1])    # True (ostatni element)

# Długość krotki
print(f"Liczba elementów: {len(dane_osoby)}")

# Sprawdzanie czy element należy do krotki
print('Anna' in dane_osoby)  # True
```

```
print(30 in dane_osoby)       # False

# Iteracja po krotce
for element in dane_osoby:
    print(element)

# Metody krotek (tylko 2!)
liczby = (1, 2, 2, 3, 2, 4, 5, 2)
print(liczby.count(2))   # 4 (ile razy 2 występuje)
print(liczby.index(3))   # 3 (indeks pierwszego wystąpienia 3)

# WAŻNE: Krotki są niemutowalne!
# dane_osoby[1] = 26  # TypeError: 'tuple' object does not support item
assignment
```

Wyjaśnienie:

- Krotki tworzymy używając nawiasów okrągłych `()`
- Jednoelementowa krotka wymaga przecinka: `(5,)` nie `(5)`
- Dostęp do elementów jak w listach: przez indeksy i slicing
- Krotki mają tylko 2 metody: `.count()` i `.index()`
- **Niemutowalność** - nie można zmienić zawartości po utworzeniu

Przykład 2: Tuple unpacking i zwracanie wielu wartości

```
# Tuple unpacking - rozpakowywanie krotek do zmiennych
punkt = (3, 5)
x, y = punkt  # Unpacking
print(f"x = {x}, y = {y}")

# Przykład z danymi osoby
osoba = ('Jan', 'Kowalski', 30, 'Warszawa')
imie, nazwisko, wiek, miasto = osoba
print(f"{imie} {nazwisko}, {wiek} lat, mieszka w {miasto}")

# Zamiana wartości bez zmiennej pomocniczej (dzięki tuple unpacking!)
a = 5
b = 10
a, b = b, a  # Zamiana wartości
print(f"a = {a}, b = {b}")  # a = 10, b = 5

# Funkcja zwracająca wiele wartości (jako krotka)
def oblicz_statystyki(liczby):
    """Zwraca średnią, minimum i maximum z listy liczb"""
```

```
    srednia = sum(liczby) / len(liczby)
    minimum = min(liczby)
    maksimum = max(liczby)
    return srednia, minimum, maksimum  # Zwraca krotkę!

# Wywołanie funkcji i unpacking
dane = [10, 20, 30, 40, 50]
avg, min_val, max_val = oblicz_statystyki(dane)
print(f"Średnia: {avg}, Min: {min_val}, Max: {max_val}")

# Unpacking z * operator (dla pozostałych elementów)
liczby = (1, 2, 3, 4, 5, 6, 7, 8, 9)
pierwszy, drugi, *reszta, ostatni = liczby
print(f"Pierwszy: {pierwszy}")        # 1
print(f"Drugi: {drugi}")               # 2
print(f"Reszta: {reszta}")             # [3, 4, 5, 6, 7, 8]
print(f"Ostatni: {ostatni}")           # 9

# Pominięcie niepotrzebnych wartości z _
imie, _, wiek, _ = osoba  # Interesują nas tylko imię i wiek
print(f"{imie}, {wiek} lat")
```

Wyjaśnienie:

- **Tuple unpacking** pozwala przypisać elementy krotki do osobnych zmiennych
- Funkcje często zwracają krotki by oddać wiele wartości naraz
- `*` operator zbiera pozostałe elementy do listy
- `_` (underscore) używamy dla wartości które chcemy zignorować
- Tuple unpacking to elegancki sposób na zamianę wartości zmiennych

Przykład 3: Krotki jako klucze słowników - współrzędne geograficzne

```
# Krotki jako klucze słowników - przykład: temperatura w różnych miastach
(lat, lon)
temperatury = {
    (52.2297, 21.0122): {'miasto': 'Warszawa', 'temp': 18.5},
    (50.0647, 19.9450): {'miasto': 'Kraków', 'temp': 17.2},
    (54.3520, 18.6466): {'miasto': 'Gdańsk', 'temp': 15.8},
    (51.1079, 17.0385): {'miasto': 'Wrocław', 'temp': 19.1}
}

# Dostęp przez współrzędne
warszawa_coords = (52.2297, 21.0122)
```

```python
print(temperatury[warszawa_coords])

# Iteracja po współrzędnych i danych
print("Temperatury w miastach:")
for (lat, lon), dane in temperatury.items():
    print(f"{dane['miasto']} ({lat:.2f}, {lon:.2f}): {dane['temp']}°C")

# Krotki w liście - reprezentacja datasetu jako lista rekordów
dataset_students = [
    ('Anna', 'Kowalska', 24, 4.5),
    ('Jan', 'Nowak', 22, 3.8),
    ('Maria', 'Wiśniewska', 23, 5.0),
    ('Piotr', 'Zieliński', 25, 4.2)
]

# Przetwarzanie danych
print("\nLista studentów:")
for imie, nazwisko, wiek, ocena in dataset_students:
    status = "Bardzo dobry" if ocena >= 4.5 else "Dobry" if ocena >= 4.0
else "Średni"
    print(f"{imie} {nazwisko} ({wiek} lat): {ocena} - {status}")

# Sortowanie krotek (po pierwszym elemencie, potem po drugim, itd.)
posortowani_wiek = sorted(dataset_students, key=lambda x: x[2])  # Po wieku
posortowani_ocena = sorted(dataset_students, key=lambda x: x[3],
reverse=True)  # Po ocenie malejąco

print("\nPosortowani po ocenie:")
for student in posortowani_ocena:
    print(f"{student[0]} {student[1]}: {student[3]}")

# Tuple immutability gwarantuje stabilność danych
# student = dataset_students[0]
# student[3] = 5.0  # TypeError - nie można modyfikować!
# Trzeba stworzyć nową krotkę:
dataset_students[0] = ('Anna', 'Kowalska', 24, 5.0)  # Podmiana całej krotki
```

Wyjaśnienie:

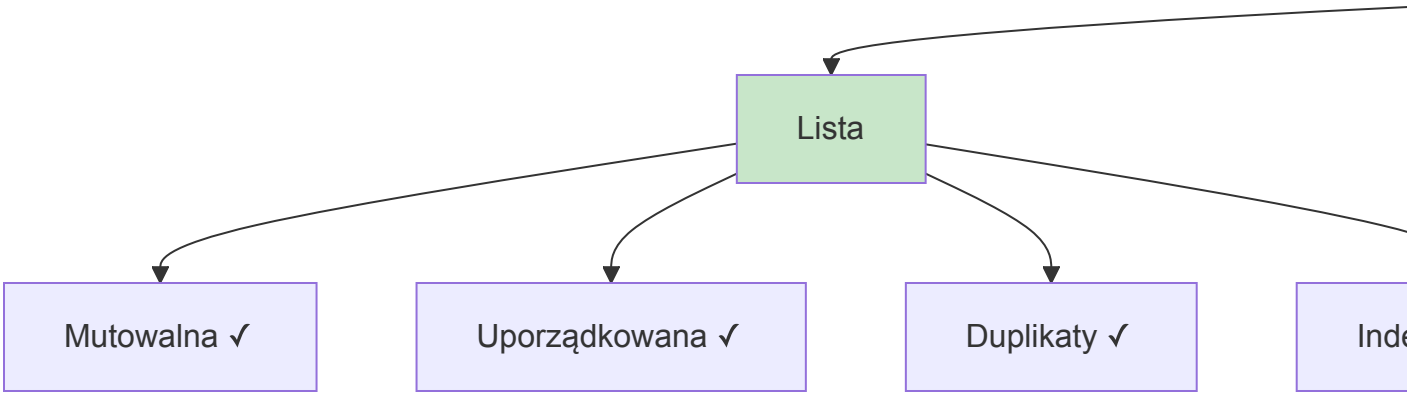
- Krotki mogą być kluczami słowników (listy nie mogą!)
- Listy krotek to dobry sposób reprezentacji tabelarycznych danych
- Sortowanie krotek odbywa się element po elemencie
- Niemutowalność krotek gwarantuje integralność danych
- Krotki są szybsze niż listy (mniejszy overhead)

Porównanie: Lista vs Krotka vs Słownik vs Zbiór

```python
# Demonstracja różnic między kolekcjami
import time

# LISTA - mutowalna, uporządkowana, duplikaty OK
lista = [1, 2, 3, 4, 5, 2, 3]
lista.append(6)  # OK
lista[0] = 10    # OK
print(f"Lista: {lista}")

# KROTKA - niemutowalna, uporządkowana, duplikaty OK
krotka = (1, 2, 3, 4, 5, 2, 3)
# krotka.append(6)  # AttributeError - brak metody append
# krotka[0] = 10    # TypeError - niemutowalna
print(f"Krotka: {krotka}")

# SŁOWNIK - mutowalny, pary klucz-wartość, klucze unikalne
slownik = {'a': 1, 'b': 2, 'c': 3}
slownik['d'] = 4  # OK
slownik['a'] = 10 # OK - nadpisanie wartości
print(f"Słownik: {slownik}")

# ZBIÓR - mutowalny, nieuporządkowany, elementy unikalne
zbior = {1, 2, 3, 4, 5, 2, 3}  # Duplikaty zostaną usunięte automatycznie
zbior.add(6)  # OK
print(f"Zbiór: {zbior}")  # {1, 2, 3, 4, 5, 6}

# Szybkość dostępu - porównanie
n = 1000000
dane_lista = list(range(n))
dane_krotka = tuple(range(n))
dane_zbior = set(range(n))

# Sprawdzanie czy element należy do kolekcji
szukany = n - 1  # Ostatni element

# Lista
start = time.time()
_ = szukany in dane_lista
czas_lista = time.time() - start

# Krotka
start = time.time()
_ = szukany in dane_krotka
czas_krotka = time.time() - start
```

```
# Zbiór
start = time.time()
_ = szukany in dane_zbior
czas_zbior = time.time() - start

print(f"\nSprawdzanie przynależności (n={n}):")
print(f"Lista: {czas_lista*1000:.4f} ms")
print(f"Krotka: {czas_krotka*1000:.4f} ms")
print(f"Zbiór: {czas_zbior*1000:.6f} ms")
print("Zbiór jest ~1000x szybszy!")
```

Schemat porównawczy kolekcji



Tip

Kiedy używać której kolekcji?

- **Lista**: Potrzebujesz uporządkowanej, mutowalnej sekwencji (np. dane czasowe)
- **Krotka**: Dane które nie powinny się zmieniać (np. współrzędne, rekordy)
- **Słownik**: Pary klucz-wartość, szybki dostęp przez klucze (np. konfiguracja, metadane)
- **Zbiór**: Unikalne elementy, szybkie sprawdzanie przynależności (np. deduplikacja)

Warning

Krotki są niemutowalne, ale jeśli zawierają mutowalne obiekty (np. listy), te obiekty mogą być modyfikowane:

```
t = (1, [2, 3], 4)
t[1].append(5)  # OK! Lista wewnątrz krotki może się zmienić
print(t)  # (1, [2, 3, 5], 4)
```

🌱 Aspekt środowiskowy

W tej lekcji zwracamy uwagę na efektywne wykorzystanie struktur danych w kontekście zużycia zasobów:

Optymalizacja pamięci

Krotki vs Listy - zużycie pamięci:

Krotki zużywają **mniej pamięci** niż listy, ponieważ są niemutowalne. Jeśli dane nie będą się zmieniać, lepiej użyć krotki.

```
import sys

# Porównanie rozmiaru w pamięci
lista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
krotka = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

rozmiar_lista = sys.getsizeof(lista)
rozmiar_krotka = sys.getsizeof(krotka)

print(f"Rozmiar listy: {rozmiar_lista} bajtów")
print(f"Rozmiar krotki: {rozmiar_krotka} bajtów")
print(f"Różnica: {rozmiar_lista - rozmiar_krotka} bajtów
({((rozmiar_lista/rozmiar_krotka - 1)*100):.1f}% więcej dla listy)")

# Dla dużych struktur różnica jest znacząca
duza_lista = list(range(100000))
duza_krotka = tuple(range(100000))

print(f"\nDuża lista: {sys.getsizeof(duza_lista) / 1024:.2f} KB")
print(f"Duża krotka: {sys.getsizeof(duza_krotka) / 1024:.2f} KB")
```

Wybór odpowiedniej struktury:

- Używaj **zbiorów** zamiast list gdy sprawdzasz przynależność elementu (szybsze!)
- Używaj **krotek** zamiast list dla danych niezmiennych (mniej pamięci)

- Używaj **dictionary comprehension** zamiast pętli (szybsze, mniej iteracji)

Efektywne algorytmy

```
# NIEEFEKTYWNE - sprawdzanie przynależności w liście
lista_kategorii = ['A', 'B', 'C', 'D', 'E'] * 1000  # 5000 elementów
def waliduj_nieefektywnie(kategoria):
    return kategoria in lista_kategorii  # O(n) - liniowe!

# EFEKTYWNE - sprawdzanie przynależności w zbiorze
zbior_kategorii = set(lista_kategorii)
def waliduj_efektywnie(kategoria):
    return kategoria in zbior_kategorii  # O(1) - stałe!

# Porównanie czasów
import time

kategoria_test = 'Z'  # Nie ma w zbiorze

# Nieefektywny sposób
start = time.time()
for _ in range(10000):
    waliduj_nieefektywnie(kategoria_test)
czas_lista = time.time() - start

# Efektywny sposób
start = time.time()
for _ in range(10000):
    waliduj_efektywnie(kategoria_test)
czas_zbior = time.time() - start

print(f"Czas walidacji z listą: {czas_lista*1000:.2f} ms")
print(f"Czas walidacji ze zbiorem: {czas_zbior*1000:.2f} ms")
print(f"Przyspieszenie: {czas_lista/czas_zbior:.1f}x")
```

Deduplikacja danych

Niepotrzebne duplikaty zajmują miejsce w pamięci i spowalniają operacje. Zbiory pozwalają szybko je usunąć.

```
# Dataset z duplikatami
dane_surowe = ['A', 'B', 'C', 'A', 'D', 'B', 'E', 'A', 'C', 'F'] * 100  #
1000 elementów

# Usunięcie duplikatów
```

```
dane_unikalne = list(set(dane_surowe))

print(f"Oryginał: {len(dane_surowe)} elementów")
print(f"Bez duplikatów: {len(dane_unikalne)} elementów")
print(f"Oszczędność pamięci: {(1 -
len(dane_unikalne)/len(dane_surowe))*100:.1f}%")
```

✓ **Success**

Green IT w kolekcjach:

1. Wybieraj najmniejszą strukturę która spełnia wymagania (krotka < lista)
2. Używaj zbiorów dla operacji sprawdzania przynależności (O(1) vs O(n))
3. Usuwaj duplikaty wcześnie by zmniejszyć rozmiar danych
4. Dictionary/set comprehension są szybsze niż pętle for

📊 Praca z Prawdziwymi Danymi

Dataset: Analiza wyników ankiety o językach programowania

W tym przykładzie przeanalizujemy dane z fikcyjnej ankiety przeprowadzonej wśród 100 programistów.

```
# Symulacja surowych danych z ankiety - 100 odpowiedzi
import random
random.seed(42)  # Dla reprodukowalności

# Generowanie odpowiedzi (język, doświadczenie_w_latach,
ocena_satysfakcji_1_10)
jezyki = ['Python', 'JavaScript', 'Java', 'C++', 'C#', 'Ruby', 'Go', 'Rust',
'PHP']

ankieta_surowa = []
for i in range(100):
    jezyk = random.choice(jezyki)
    doswiadczenie = random.randint(0, 15)
    satysfakcja = random.randint(1, 10)
    ankieta_surowa.append((jezyk, doswiadczenie, satysfakcja))

print(f"Zebrano {len(ankieta_surowa)} odpowiedzi")
print("Pierwsze 5 odpowiedzi:")
```

```
for i, odpowiedz in enumerate(ankieta_surowa[:5], 1):
    print(f"{i}. Język: {odpowiedz[0]}, Doświadczenie: {odpowiedz[1]} lat,
Satysfakcja: {odpowiedz[2]}/10")
```

Analiza z użyciem słowników

```
# Liczenie częstości języków
czestosci_jezykow = {}
suma_satysfakcji = {}
liczba_odpowiedzi = {}

for jezyk, doswiadczenie, satysfakcja in ankieta_surowa:
    # Liczenie odpowiedzi
    czestosci_jezykow[jezyk] = czestosci_jezykow.get(jezyk, 0) + 1

    # Suma satysfakcji dla średniej
    suma_satysfakcji[jezyk] = suma_satysfakcji.get(jezyk, 0) + satysfakcja
    liczba_odpowiedzi[jezyk] = liczba_odpowiedzi.get(jezyk, 0) + 1

# Obliczanie średniej satysfakcji
srednia_satysfakcja = {}
for jezyk in suma_satysfakcji:
    srednia_satysfakcja[jezyk] = suma_satysfakcji[jezyk] /
liczba_odpowiedzi[jezyk]

# Raport
print("\n=== RAPORT Z ANKIETY ===")
print(f"Liczba języków w ankiecie: {len(czestosci_jezykow)}")
print(f"\nCzęstość wyboru języków:")

# Sortowanie po liczbie odpowiedzi
posortowane = sorted(czestosci_jezykow.items(), key=lambda x: x[1],
reverse=True)

for jezyk, liczba in posortowane:
    procent = (liczba / len(ankieta_surowa)) * 100
    srednia = srednia_satysfakcja[jezyk]
    print(f"  {jezyk}: {liczba} odpowiedzi ({procent:.1f}%), średnia
satysfakcja: {srednia:.2f}/10")

# Znajdowanie języka z najwyższą satysfakcją
najlepsza_satysfakcja = max(srednia_satysfakcja.items(), key=lambda x: x[1])
print(f"\nJęzyk z najwyższą satysfakcją: {najlepsza_satysfakcja[0]}
({najlepsza_satysfakcja[1]:.2f}/10)")
```

Analiza z użyciem zbiorów

```
# Unikalne języki (deduplikacja)
wszystkie_jezyki = [jezyk for jezyk, _, _ in ankieta_surowa]
unikalne_jezyki = set(wszystkie_jezyki)
print(f"\nUnikalne języki w ankiecie: {unikalne_jezyki}")
print(f"Liczba unikalnych języków: {len(unikalne_jezyki)}")

# Filtrowanie danych - języki z wysoką satysfakcją (>= 8/10)
jezyki_wysoka_satysfakcja = set()
for jezyk, _, satysfakcja in ankieta_surowa:
    if satysfakcja >= 8:
        jezyki_wysoka_satysfakcja.add(jezyk)

print(f"\nJęzyki z wysoką satysfakcją (>=8): {jezyki_wysoka_satysfakcja}")

# Języki z niską satysfakcją (<= 3/10)
jezyki_niska_satysfakcja = set()
for jezyk, _, satysfakcja in ankieta_surowa:
    if satysfakcja <= 3:
        jezyki_niska_satysfakcja.add(jezyk)

# Operacje zbiorowe
tylko_wysoka = jezyki_wysoka_satysfakcja - jezyki_niska_satysfakcja
tylko_niska = jezyki_niska_satysfakcja - jezyki_wysoka_satysfakcja
obie_kategorie = jezyki_wysoka_satysfakcja & jezyki_niska_satysfakcja

print(f"Tylko wysokie oceny: {tylko_wysoka}")
print(f"Tylko niskie oceny: {tylko_niska}")
print(f"W obu kategoriach: {obie_kategorie}")
```

Wizualizacja wyników

```
import matplotlib.pyplot as plt

# Przygotowanie danych do wizualizacji
jezyki_lista = []
liczby_lista = []
satysfakcje_lista = []

for jezyk, liczba in posortowane:
    jezyki_lista.append(jezyk)
    liczby_lista.append(liczba)
    satysfakcje_lista.append(srednia_satysfakcja[jezyk])
```

```
# Wykres 1: Popularność języków
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.bar(jezyki_lista, liczby_lista, color='steelblue', alpha=0.8)
plt.xlabel('Język programowania')
plt.ylabel('Liczba odpowiedzi')
plt.title('Popularność języków programowania')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', alpha=0.3)

# Wykres 2: Średnia satysfakcja
plt.subplot(1, 2, 2)
colors = ['green' if s >= 7 else 'orange' if s >= 5 else 'red' for s in
satysfakcje_lista]
plt.bar(jezyki_lista, satysfakcje_lista, color=colors, alpha=0.8)
plt.xlabel('Język programowania')
plt.ylabel('Średnia satysfakcja (1-10)')
plt.title('Średnia satysfakcja programistów')
plt.xticks(rotation=45, ha='right')
plt.axhline(y=5, color='gray', linestyle='--', alpha=0.5, label='Średnia
(5.0)')
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.ylim(0, 10)

plt.tight_layout()
plt.savefig('analiza_jezykow.png', dpi=150, bbox_inches='tight')
print("\nWykresy zapisane do pliku: analiza_jezykow.png")
# plt.show()  # Odkomentuj by wyświetlić
```

🧪 Zadania do samodzielnej pracy

✏️ Zadania podstawowe (1-8)

1. ✏️ **Zadanie 1 – Tworzenie słownika z informacjami o osobie**

   Stwórz słownik `osoba` zawierający klucze: `imie`, `nazwisko`, `wiek`, `miasto`, `email`. Wypełnij go przykładowymi danymi i wypisz wszystkie informacje w czytelny sposób.

   Wymagania:

   - Użyj f-stringów do formatowania

- Wypisz każdą informację w osobnej linii

Oczekiwany wynik:

- Wydruk typu: "Imię: Jan, Nazwisko: Kowalski, ..."

(proste)

2. ✏️ **Zadanie 2 – Dodawanie i usuwanie elementów ze słownika**

   Masz słownik z produktami i cenami: `{'jabłko': 3.50, 'banan': 2.20, 'pomarańcza': 4.00}`. Dodaj nowy produkt `'gruszka': 3.80`, usuń `'banan'`, zmień cenę jabłka na `3.20` i wypisz zaktualizowany słownik.

   Wymagania:

   - Użyj odpowiednich operacji na słowniku
   - Wypisz słownik przed i po zmianach

   (proste)

3. ✏️ **Zadanie 3 – Liczenie wystąpień słów**

   Masz listę słów: `['python', 'java', 'python', 'javascript', 'python', 'java', 'c++', 'python']`. Policz ile razy każde słowo występuje używając słownika.

   Wymagania:

   - Użyj pętli i metody `.get()`
   - Wypisz wyniki

   Oczekiwany wynik:

   - Słownik z częstościami: `{'python': 4, 'java': 2, ...}`

   (proste)

4. ✏️ **Zadanie 4 – Operacje na zbiorach**

   Masz dwa zbiory: `A = {1, 2, 3, 4, 5}` i `B = {4, 5, 6, 7, 8}`. Oblicz i wypisz:

   - Sumę zbiorów
   - Przecięcie
   - Różnicę A - B
   - Różnicę symetryczną

   Wymagania:

- Użyj operatorów zbiorowych (`|` , `&` , `-` , `^`)
- Wypisz każdy wynik z opisem

(proste)

5. ✏️ **Zadanie 5 – Usuwanie duplikatów z listy**

   Masz listę z duplikatami: `[1, 2, 3, 2, 4, 1, 5, 3, 6, 4, 7]` . Usuń duplikaty używając zbioru i przekonwertuj z powrotem na listę. Wypisz obie listy i porównaj długości.

   Wymagania:

   - Użyj `set()` do usunięcia duplikatów
   - Wypisz przed i po

   (proste)

6. ✏️ **Zadanie 6 – Tworzenie krotek i dostęp do elementów**

   Stwórz krotkę z danymi studenta: `(imię, nazwisko, wiek, średnia_ocen)` . Użyj tuple unpacking by przypisać każdy element do osobnej zmiennej i wypisz informacje.

   Wymagania:

   - Stwórz krotkę z przykładowymi danymi
   - Użyj unpacking
   - Wypisz sformatowany wynik

   (proste)

7. ✏️ **Zadanie 7 – Zwracanie wielu wartości z funkcji**

   Napisz funkcję `analiza_liczb(lista)` która zwraca krotkę z: sumą, średnią, minimum i maksimum z listy. Przetestuj na liście `[10, 20, 30, 40, 50]` .

   Wymagania:

   - Funkcja zwraca krotkę z 4 wartościami
   - Użyj tuple unpacking przy wywołaniu
   - Wypisz wyniki

   (proste)

8. ✏️ **Zadanie 8 – Porównanie kolekcji**

   Stwórz te same dane (liczby od 1 do 10) jako listę, krotkę i zbiór. Sprawdź czy `5` należy do każdej z nich i zmierz czas używając `time.time()` .

   Wymagania:

- Stwórz 3 kolekcje z tymi samymi danymi
- Sprawdź `5 in kolekcja` dla każdej
- Porównaj (niekoniecznie precyzyjnie) szybkość

(proste)

✏️ Zadania średnie (9-12)

9. ✏️ **Zadanie 9 – Inwersja słownika**

   Masz słownik mapujący nazwy produktów na kategorie: `{'laptop': 'elektronika', 'klawiatura': 'elektronika', 'krzesło': 'meble', 'biurko': 'meble'}`. Stwórz odwrotny słownik gdzie klucze to kategorie, a wartości to listy produktów.

   Wymagania:

   - Iteruj po oryginalnym słowniku
   - Zgrupuj produkty według kategorii
   - Wynik: `{'elektronika': ['laptop', 'klawiatura'], 'meble': [...]}`

   Oczekiwany wynik:

   - Słownik `kategoria: lista_produktów`

   (średnie)

10. ✏️ **Zadanie 10 – Walidacja danych ze zbiorem dozwolonych wartości**

    Masz listę kategorii od użytkowników: `['sport', 'muzyka', 'SPORT', 'film', 'xyz', 'nauka', 'gaming']`. Dozwolone kategorie to `{'sport', 'muzyka', 'film', 'nauka', 'gry'}`. Znajdź nieprawidłowe kategorie (uwzględnij wielkość liter - konwertuj na małe).

    Wymagania:

    - Konwertuj wszystkie kategorie na małe litery
    - Użyj operacji zbiorowych by znaleźć nieprawidłowe
    - Wypisz prawidłowe i nieprawidłowe osobno

    (średnie)

11. ✏️ **Zadanie 11 – Dictionary comprehension - kwadraty liczb**

    Stwórz słownik używając dictionary comprehension, gdzie klucze to liczby od 1 do 20, a wartości to ich kwadraty. Następnie przefiltruj ten słownik (też comprehension!) by zawierał tylko liczby parzyste.

    Wymagania:

- Użyj dictionary comprehension 2 razy
- Pierwszy słownik: wszystkie liczby 1-20
- Drugi słownik: tylko parzyste klucze

Oczekiwany wynik:

- Dwa słowniki wypisane

(średnie)

12. ✏️ **Zadanie 12 – Analiza transakcji - słowniki zagnieżdżone**

Masz listę transakcji (krotek): `[('2024-01-15', 'Elektronika', 1200), ('2024-01-16', 'Meble', 800), ('2024-01-15', 'Elektronika', 350), ...]`. Stwórz słownik gdzie kluczem jest data, a wartością słownik z sumą sprzedaży dla każdej kategorii tego dnia.

Dataset:

```
transakcje = [
    ('2024-01-15', 'Elektronika', 1200),
    ('2024-01-15', 'Meble', 450),
    ('2024-01-16', 'Elektronika', 800),
    ('2024-01-16', 'Elektronika', 350),
    ('2024-01-15', 'Odzież', 200),
    ('2024-01-16', 'Meble', 600)
]
```

Wymagania:

- Zagnieżdżone słowniki: `{data: {kategoria: suma}}`
- Iteruj po transakcjach i akumuluj sumy

Oczekiwany wynik:

- Słownik typu: `{'2024-01-15': {'Elektronika': 1200, 'Meble': 450, ...}, ...}`

(średnie)

🧠 Zadania wyzwanie (13-20)

13. 🧠 **Zadanie 13 – Analiza sentymentu komentarzy**

Masz listę komentarzy (stringi) i słownik słów pozytywnych/negatywnych. Przeanalizuj każdy komentarz i określ czy jest pozytywny (więcej słów pozytywnych), negatywny czy neutralny.

Dataset:

```
komentarze = [
    "Ten produkt jest świetny i fantastyczny",
    "Okropny towar, kiepska jakość",
    "Całkiem dobry, ale mogłoby być lepiej",
    "Rewelacja, super jakość, polecam",
    "Słaby produkt, nie polecam"
]

slowa_pozytywne = {'świetny', 'fantastyczny', 'dobry', 'super',
'rewelacja', 'polecam'}
slowa_negatywne = {'okropny', 'kiepski', 'słaby', 'nie polecam', 'zły'}
```

Wymagania:

- Rozdziel komentarze na słowa (użyj `.lower().split()`)
- Policz słowa pozytywne i negatywne w każdym komentarzu
- Określ sentyment (pozytywny/negatywny/neutralny)
- Zwróć listę krotek: `(komentarz, sentyment, liczba_poz, liczba_neg)`

Oczekiwany wynik:

- Lista z analizą sentymentu dla każdego komentarza
- Statystyka: ile komentarzy pozytywnych/negatywnych/neutralnych

(challenge)

14. 🧠 **Zadanie 14 – Grupowanie studentów według średniej ocen**

Masz listę krotek ze studentami: `(imię, nazwisko, ocena1, ocena2, ocena3)`. Pogrupuj studentów w słowniku według kategorii: 'wyróżniający' (średnia >= 4.5), 'dobry' (>= 3.5), 'średni' (< 3.5).

Dataset:

```
studenci = [
    ('Anna', 'Kowalska', 5.0, 4.5, 5.0),
    ('Jan', 'Nowak', 3.0, 3.5, 3.0),
    ('Maria', 'Zielińska', 4.0, 4.5, 5.0),
    ('Piotr', 'Wiśniewski', 2.0, 3.0, 2.5),
    ('Ewa', 'Wójcik', 5.0, 5.0, 4.5),
    ('Tomasz', 'Kamiński', 4.0, 3.5, 4.0)
]
```

Wymagania:

- Oblicz średnią z 3 ocen dla każdego studenta
- Stwórz słownik `{kategoria: lista_studentów}`
- Każdy student jako krotka `(imię, nazwisko, średnia)`
- Wypisz pogrupowanych studentów

Oczekiwany wynik:

- Słownik z 3 kategoriami i listami studentów

(challenge)

15. 🧠 **Zadanie 15 – Znajdowanie przecięć i różnic w zbiorach danych**

Masz 3 zbiory użytkowników którzy odwiedzili stronę w różnych dniach. Znajdź: użytkowników którzy byli wszystkie 3 dni, tylko jeden dzień, dokładnie 2 dni.

Dataset:

```
dzien1 = {'user001', 'user002', 'user003', 'user004', 'user005',
'user010'}
dzien2 = {'user002', 'user003', 'user006', 'user007', 'user008',
'user010'}
dzien3 = {'user003', 'user004', 'user009', 'user010', 'user011'}
```

Wymagania:

- Użyj operacji zbiorowych
- Znajdź użytkowników aktywnych wszystkie 3 dni
- Znajdź użytkowników tylko w 1 dniu (w każdym dniu osobno)
- Znajdź użytkowników w dokładnie 2 dniach

Oczekiwany wynik:

- Zestawienie użytkowników według liczby dni aktywności
- Szczegółowy raport

(challenge)

16. 🧠 **Zadanie 16 – Cache funkcji używając słownika (memoization)**

Napisz funkcję `fibonacci(n)` która oblicza n-tą liczbę Fibonacciego. Użyj słownika jako cache by przechowywać już obliczone wartości (memoization). Porównaj czas wykonania z i bez cache dla `n=35`.

Wymagania:

- Implementacja rekurencyjna z cache
- Cache jako słownik globalny lub parametr z wartością domyślną
- Pomiar czasu dla obu wersji
- Wypisz liczbę wywołań funkcji

Oczekiwany wynik:

- Fibonacci(35) obliczone
- Porównanie czasu: z cache vs bez cache
- Demonstracja przyspieszenia

(challenge)

17. 🧠 **Zadanie 17 – Inwersja wielu-do-wielu - słownik z setami**

Masz słownik gdzie jeden film może mieć wiele gatunków: `{'Film1': ['Akcja', 'Sci-Fi'], 'Film2': ['Komedia'], ...}`. Stwórz odwrotny słownik: `{gatunek: zbiór_filmów}`.

Dataset:

```
filmy_gatunki = {
    'Matrix': ['Akcja', 'Sci-Fi'],
    'Shrek': ['Komedia', 'Familijny', 'Animacja'],
    'Inception': ['Akcja', 'Sci-Fi', 'Thriller'],
    'Toy Story': ['Animacja', 'Familijny'],
    'Die Hard': ['Akcja', 'Thriller'],
    'The Hangover': ['Komedia']
}
```

Wymagania:

- Klucze: gatunki, wartości: zbiory filmów (set!)
- Iteruj po filmach i dodawaj do odpowiednich zbiorów
- Wypisz każdy gatunek i filmy które do niego należą

Oczekiwany wynik:

- Słownik: `{gatunek: {film1, film2, ...}}`

(challenge)

18. 🧠 **Zadanie 18 – Analiza podobieństwa zestawów produktów - Jaccard similarity**

Masz 3 koszyki zakupów (zbiory produktów). Oblicz podobieństwo Jaccarda między każdą parą koszyków. Jaccard = |A ∩ B| / |A ∪ B|

Dataset:

```
koszyk1 = {'mleko', 'chleb', 'masło', 'jajka', 'ser'}
koszyk2 = {'mleko', 'chleb', 'dżem', 'jogurt', 'ser'}
koszyk3 = {'woda', 'sok', 'chipsy', 'czekolada'}
```

Wymagania:

- Napisz funkcję `jaccard_similarity(set1, set2)` zwracającą podobieństwo (0-1)
- Oblicz dla wszystkich par: (1,2), (1,3), (2,3)
- Interpretuj wyniki (0 = brak wspólnych, 1 = identyczne)

Oczekiwany wynik:

- Podobieństwa między koszykami
- Interpretacja: które koszyki są najbardziej podobne?

(challenge)

19. 🧠 **Zadanie 19 – Word frequency analysis - top N słów**

Przeanalizuj długi tekst i znajdź 10 najczęściej występujących słów. Użyj słownika do zliczania i zbioru do odfiltrowania stop words (częste słowa bez znaczenia).

Dataset:

```
tekst = """
Python jest językiem programowania który jest bardzo popularny w data
science.
Python jest łatwy do nauki i ma ogromną społeczność. Wiele firm używa
Python
do analizy danych i machine learning. Python jest uniwersalny i można go
używać
do web development, automatyzacji i wielu innych zastosowań. Python jest
open source.
"""

stop_words = {'jest', 'do', 'i', 'w', 'można', 'go', 'się', 'z', 'na',
'po'}
```

Wymagania:

- Konwertuj tekst na małe litery, podziel na słowa
- Usuń stop words używając zbioru
- Policz częstości słów (słownik)
- Posortuj i wypisz top 10

Oczekiwany wynik:

- Lista 10 najczęstszych słów z liczbą wystąpień
- Wykres słupkowy (matplotlib) - opcjonalnie

(challenge)

20. 🧠 **Zadanie 20 – Dataset deduplication z zachowaniem kolejności**

Masz listę rekordów (krotek) z danymi użytkowników. Niektóre rekordy są duplikatami. Usuń duplikaty używając zbioru, ale zachowaj kolejność pierwszego wystąpienia (zbiory nie zachowują kolejności!).

Dataset:

```
uzytkownicy = [
    ('user001', 'Anna', 'Kowalska'),
    ('user002', 'Jan', 'Nowak'),
    ('user001', 'Anna', 'Kowalska'),  # duplikat
    ('user003', 'Maria', 'Zielińska'),
    ('user002', 'Jan', 'Nowak'),      # duplikat
    ('user004', 'Piotr', 'Wiśniewski'),
    ('user003', 'Maria', 'Zielińska'), # duplikat
    ('user005', 'Ewa', 'Wójcik')
]
```

Wymagania:

- Usuń duplikaty (po pierwszym polu - ID użytkownika)
- Zachowaj kolejność pierwszego wystąpienia
- Użyj zbioru do śledzenia już widzianych ID
- Wypisz oryginalne rekordy i po deduplikacji
- Policz ile duplikatów usunięto

Oczekiwany wynik:

- Lista bez duplikatów w oryginalnej kolejności
- Raport: ile rekordów usunięto, oszczędność pamięci

(challenge)

📚 Podsumowanie

W tej lekcji poznaliśmy trzy zaawansowane kolekcje Pythona - **słowniki**, **zbiory** i **krotki** - które są fundamentalnymi narzędziami w Data Science:

Kluczowe punkty:

Słowniki (dict):

- Przechowują pary klucz-wartość
- Szybki dostęp do wartości przez klucze
- Idealne do mapowania, agregacji, metadanych
- Metody: `.get()`, `.keys()`, `.values()`, `.items()`, `.update()`
- Dictionary comprehension dla zwięzłego kodu

Zbiory (set):

- Przechowują unikalne elementy
- Bardzo szybkie sprawdzanie przynależności (O(1))
- Operacje matematyczne: suma, przecięcie, różnica
- Idealne do deduplikacji i porównań grup danych
- Nie zachowują kolejności

Krotki (tuple):

- Niemutowalne, uporządkowane sekwencje
- Zużywają mniej pamięci niż listy
- Mogą być kluczami słowników
- Tuple unpacking dla eleganckich przypisań
- Gwarantują integralność danych

Wybór odpowiedniej kolekcji:

- **Lista**: mutowalność + uporządkowanie + indeksowanie
- **Krotka**: niemutowalność + uporządkowanie + mniejsza pamięć
- **Słownik**: pary klucz-wartość + szybki dostęp po kluczu
- **Zbiór**: unikalność + szybkie operacje przynależności

Green IT:

- Krotki zużywają mniej pamięci niż listy
- Zbiory są szybsze przy sprawdzaniu przynależności
- Deduplikacja oszczędza pamięć i przyspiesza operacje
- Wybór odpowiedniej struktury = mniej zasobów

Te kolekcje będą intensywnie używane w kolejnych lekcjach podczas pracy z bibliotekami NumPy i Pandas, gdzie efektywne operacje na danych są kluczowe dla wydajności.

📖 Dodatkowe Zasoby

Dokumentacja Python

- Python Dictionary - oficjalna dokumentacja
- Python Sets - operacje na zbiorach
- Python Tuples - niemutowalne sekwencje

Tutoriale i Kursy

- Real Python - Dictionaries - szczegółowy przewodnik po słownikach
- Real Python - Sets - praktyczne zastosowania zbiorów
- Python Documentation - Data Structures - kompletny przewodnik

Praktyka

- LeetCode - zadania algorytmiczne używające tych struktur
- HackerRank - Python - praktyczne ćwiczenia
- Codewars - kata z użyciem kolekcji

Zaawansowane tematy

- Time Complexity - złożoność czasowa operacji na kolekcjach
- Collections module - zaawansowane struktury (Counter, defaultdict, OrderedDict)

Green IT

- Green Software Foundation - praktyki zrównoważonego programowania
- Energy Efficient Python - badania nad efektywnością energetyczną

Gratulacje! 🎉 Ukończyłeś lekcję o kolekcjach Python. Teraz znasz wszystkie podstawowe struktury danych i jesteś gotowy do pracy z bibliotekami Data Science takimi jak NumPy i Pandas!